

BCR-6605 Part E – relearn, in full

Everything that has to change so a deviation on a learned threshold launches a relearn, in one document. Fifteen steps in three phases that ship independently of each other. Every step is given as the exact text to find and the text to put in its place.

The three phases

Phase	What it does	Depends on
A 7 steps	A deviation on a threshold the configuration already declares LEARNED launches a relearn, on create and on delete	Nothing. Ready to build.
B 1 step	A deviated min or max clamps the learned value, as slide 17 requires. Closes a silent gap left by Part D.	Part D only, which is already in.
C 7 steps	A deviation that turns a FIXED threshold into a learned one – the second half of Benoit's first bullet	Phase A. Two files still needed for exact anchors.

Build them in that order. A is small and confined to the deviation screens and the business layer. B is one method and fixes a wrong answer rather than a missing feature, so it is worth doing even if C slips. C is the large one and touches the alarm path for every aircraft.

What the requirement says

Synthesis document, *Deviation creation* rules: “A deviation set on a learned threshold triggers an automatic relearn request.” Slide 8: “A deviation is not deleted by a relearn request.” Slide 15: the relearn is *sur les heures suivantes* – following hours only, so no past-flight selection and no applicable date.

Slide 17 gives the matrix, and it covers deletion as well as creation:

Deviation	On create	On delete
FIXED → LEARNED	relearn	–
LEARNED → LEARNED, bounds changed	relearn	relearn
LEARNED → FIXED	–	relearn

Every row reduces to one sentence: **launch a relearn whenever the threshold is LEARNED after the change**. On create that is the deviated type; on delete it is the configuration type, because the deviation is gone.

Benoit's note of 30-09 pins down the detail:

He says	Means
if the indicator is LEARNED (type learned or modified from fixed to learned)	Both cases, including a deviation that turns a FIXED threshold into a learned one – see <i>Fixed to learned</i> at the end
Programmatically launch the relearn request without showing the popup	No Threshold Learning dialog; the request is built in code
Description: fill with “New deviation – TE-2026xxxxx”	The TE reference of that deviation goes in the text, so the description differs per deviation
No Flight Hours (to set the learning on future flights)	<code>historyLearningTime = 0</code>

One relearn per deviation, not one per Apply. Because the description carries the TE reference, `createRelearnThresholdRequests` has to be called once for each deviation rather than once for the whole list. That is why the launcher below takes a description and is called inside the loop.

History learning time is zero. `RelearnThresholdRequest.historyLearningTime = 0` makes `LearnedThresholdAnalyzer` skip the *flightsBefore* loop entirely, which is exactly what slide 15 describes.

Where phase A goes

`HealthCommonBusiness` already holds both call sites and already has `createRelearnThresholdRequests` on the same interface, which runs the whole server path including the data-collection record. Reusing it means no new service method and no new DAO code.

Two values have to reach those two methods and currently do not: the aircraft id and the login name. The description is not one of them – it is built from the deviation's own TE reference, so it belongs in the business layer beside the rest of the mapping.

Phase A – relearn on a configured learned threshold

Step	File	What
A1	<code>IHealthCommonBusiness</code>	Two new parameters on two methods
A2	<code>HealthCommonBusiness</code>	The helper that finds the learned thresholds, and the launcher
A3	<code>HealthCommonBusiness</code>	<code>createDeviations</code> – relearn on create
A4	<code>HealthCommonBusiness</code>	<code>acknowledgeDeviations</code> – relearn on delete
A5	<code>HealthAlarmsPreferenceStore</code>	Pass aircraft and login through
A6	<code>DeviationTableComposite</code>	Give the Delete button the aircraft
A7	<code>HealthCommonBusiness</code>	An unrelated NPE in <code>toDtos</code> , while you are in the file

A1, A3 and A4 go in together – the signature change ripples, so none of them compiles alone. A5 and A6 then fix the call sites.

Step A1 — IHealthCommonBusiness

mgs-business/.../business/health/api/IHealthCommonBusiness.java

FIND the two declarations (search for createDeviations):

```
void createDeviations(List<IndicatorDeviationDto> dtos);

void acknowledgeDeviations(List<String> deviationIds);
```

REPLACE WITH:

```
void createDeviations(String aircraftId, String loginName, List<IndicatorDeviationDto> dtos)
    throws EntityNotFoundException, ConcurrentAccessException;

void acknowledgeDeviations(String aircraftId, String loginName, List<String> deviationIds)
    throws EntityNotFoundException, ConcurrentAccessException;
```

If the current declarations already carry a throws clause, keep it and add the two above to it. Both exceptions come from createRelearnThresholdRequests, which is declared four methods further up in the same interface.

Step A2 — HealthCommonBusiness: the helper

mgs-business/.../business/health/impl/HealthCommonBusiness.java

FIND the closing brace of acknowledgeDeviations, which is immediately followed by toDtos:

```
private List<IndicatorDeviationDto> toDtos(List<IndicatorDeviation> deviations) {
```

INSERT ABOVE IT, these two methods:

```
/**
 * BCR-6605 the learned thresholds that must be relearned because of this deviation.
 *
 * @param deviatedType
 *      the threshold type after the change: the deviation's type when it set one,
 *      null to mean "whatever the configuration says" - used when the deviation is
 *      deleted, and when it did not change the type
 */
private List<String> getLearnedThresholdIds(IndicatorConfiguration indicatorConfiguration,
    String severity, String deviatedType) {
    List<String> ids = new ArrayList<>();
    if (indicatorConfiguration == null) {
        return ids;
    }
    if (deviatedType != null && !ThresholdType.LEARNED.name().equals(deviatedType)) {
        // The deviation turns the threshold into a fixed one: nothing to learn
        return ids;
    }
    for (ThresholdConfiguration threshold : indicatorConfiguration.getThresholdConfigurations()) {
        if (!(threshold instanceof LearnedThresholdConfiguration)) {
            // Only a configured learned threshold can carry a relearn request
            continue;
        }
        if (severity != null
            && !severity.equals(threshold.getThreshold().getSeverity().name())) {
            continue;
        }
        ids.add(threshold.getId());
    }
    return ids;
}

/**
 * BCR-6605 launch the relearn for one deviation, on the following flight hours only.
 * The description carries that deviation's TE reference, so this is called once per
 * deviation rather than once for the whole Apply.
 */
private void launchRelearn(String aircraftId, String loginName, List<String> thresholdIds,
    String description) throws ConcurrentAccessException {
    if (thresholdIds.isEmpty()) {
        return;
    }
    // History learning time is 0: the learning uses the following flight hours only
    createRelearnThresholdRequests(aircraftId, thresholdIds, 0L, description, loginName);
}
```

The id is the threshold configuration id. getId() on the LearnedThresholdConfiguration itself, not getThreshold().getId(). This matches what the existing Learn Threshold path passes: HealthTreeNodeLearnThresholdAction.learnThreshold() builds its learnedThresholdIds with threshold.getId() over a list of LearnedThresholdConfiguration. The wrong one fails silently – no error, just no relearn row.

No new imports. ArrayList, List, ThresholdType, ThresholdConfiguration, LearnedThresholdConfiguration, IndicatorConfiguration and ConcurrentAccessException are all already imported in this file.

Step A3 — HealthCommonBusiness: relearn on create

mgs-business/.../business/health/impl/HealthCommonBusiness.java

FIND the whole method (the first two lines and the last three are enough to locate it):

```
@Override
public void createDeviations(List<IndicatorDeviationDto> dtos) {
    for (IndicatorDeviationDto dto : dtos) {
        IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
        d.setIndicatorConfiguration(healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId()));
        ...
    }
}
```

```

        d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
        healthCommonService.createDeviation(d);
    }
}

```

REPLACE WITH:

```

@Override
public void createDeviations(String aircraftId, String loginName, List<IndicatorDeviationDto> dtos)
    throws EntityNotFoundException, ConcurrentAccessException {
    for (IndicatorDeviationDto dto : dtos) {
        // BCR-6605 hoisted into a local so the relearn lookup can reuse it
        IndicatorConfiguration indicatorConfiguration =
            healthCommonService.findIndicatorConfiguration(dto.getIndicatorConfigurationId());

        IndicatorDeviation d = new IndicatorDeviation(UUID.randomUUID().toString());
        d.setIndicatorConfiguration(indicatorConfiguration);
        d.setFilteringLaw(healthCommonService.findFilteringLaw(dto.getFilteringLawId()));
        d.setTeReference(dto.getTeReference());
        d.setStartDate(dto.getStartDate());
        d.setEndDate(null);
        d.setSeverity(dto.getSeverity());
        d.setThresholdLevel(dto.getThresholdLevel());
        d.setThresholdType(dto.getThresholdType());
        d.setThresholdValue(dto.getThresholdValue());
        d.setMinValue(dto.getMinValue());
        d.setMaxValue(dto.getMaxValue());
        d.setLearningTime(dto.getLearningTime());
        d.setMinimumDeviation(dto.getMinimumDeviation());
        d.setMeanFactor(dto.getMeanFactor());
        d.setDeviationCoefficient(dto.getDeviationCoefficient());
        d.setNbSuccessivePoints(dto.getNbSuccessivePoints());
        healthCommonService.createDeviation(d);

        // BCR-6605 the threshold is learned after this deviation, so relearn it.
        // The description carries this deviation's TE, so launch inside the loop.
        launchRelearn(aircraftId, loginName,
            getLearnedThresholdIds(indicatorConfiguration, dto.getSeverity(),
                dto.getThresholdType()),
            RELEARN_DESCRIPTION_CREATE + dto.getTeReference());
    }
}

```

Only two things changed in the body: `findIndicatorConfiguration` moved into a local so it can be used twice, and the `launchRelearn` call at the bottom of the loop. The description comes out as `New deviation - TE-2026-EC155-00010`, which is the format Benoit asked for.

And ADD the two constants at the top of the class, beside the fields:

```

/** BCR-6605 relearn description prefixes. */
private static final String RELEARN_DESCRIPTION_CREATE = "New deviation - ";
private static final String RELEARN_DESCRIPTION_DELETE = "Deviation deleted - ";

```

Benoit specified the create text. He said nothing about the delete side, so `Deviation deleted - TE-...` is a guess that follows the same shape – change the constant if he wants something else. It is one line.

Step A4 — HealthCommonBusiness: relearn on delete

mgs-business/.../business/health/impl/HealthCommonBusiness.java

FIND:

```

@Override
public void acknowledgeDeviations(List<String> deviationIds) {
    Date now = new Date();
    for (String id : deviationIds) {
        healthCommonService.acknowledgeDeviation(id, now);
    }
}

```

REPLACE WITH:

```

@Override
public void acknowledgeDeviations(String aircraftId, String loginName, List<String> deviationIds)
    throws EntityNotFoundException, ConcurrentAccessException {
    // BCR-6605 read them while they are still active: once closed they leave this list
    List<IndicatorDeviation> closing = new ArrayList<>();
    for (IndicatorDeviation deviation
        : healthCommonService.findActiveDeviations(getIndicatorIds(aircraftId))) {
        if (deviationIds.contains(deviation.getId())) {
            closing.add(deviation);
        }
    }

    Date now = new Date();
    for (String id : deviationIds) {
        healthCommonService.acknowledgeDeviation(id, now);
    }

    // BCR-6605 the threshold reverts to the configuration, so relearn if that one is learned.
    // deviatedType is null on purpose: the deviation is going away.
    for (IndicatorDeviation deviation : closing) {
        launchRelearn(aircraftId, loginName,
            getLearnedThresholdIds(deviation.getIndicatorConfiguration(),
                deviation.getSeverity(), null),
            RELEARN_DESCRIPTION_DELETE + deviation.getTeReference());
    }
}

```

There is no single-deviation lookup on the service, so this reuses `findActiveDeviations` – the same call the Deviation List tab already makes. One extra query per Delete over a handful of rows; the synthesis document puts the total at three or four per aircraft.

Step A5 — HealthAlarmsPreferenceStore

`mgs-client/.../preferences/healthalarms/store/HealthAlarmsPreferenceStore.java`

Two places. First, `saveDeviations` needs to remember which aircraft it is working on.

FIND:

```
public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    Date now = new Date();
    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
        Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
```

REPLACE WITH:

```
public void saveDeviations() {
    List<IndicatorDeviationDto> newDeviations = new ArrayList<>();
    Aircraft deviationAircraft = null;    // BCR-6605
    Date now = new Date();
    for (Entry<Aircraft, WCMechanicalComponentModel> entry : data.entrySet()) {
        deviationAircraft = entry.getKey();    // BCR-6605
        Multimap<Indicator, WCThresholdModel> thresholds = getComponentThresholds(entry.getValue());
```

Then FIND, at the end of the same method:

```
        if (!newDeviations.isEmpty()) {
            try {
                healthBusiness.createDeviations(newDeviations);
            } catch (Exception e) {
                LOGGER.error("Could not save deviations", e);
            }
        }
    }
}
```

REPLACE WITH:

```
        if (!newDeviations.isEmpty() && deviationAircraft != null) {
            try {
                healthBusiness.createDeviations(deviationAircraft.getId(),
                    MgsSession.getInstance().getUser().getLoginName(), newDeviations);
            } catch (Exception e) {
                LOGGER.error("Could not save deviations", e);
            }
        }
    }
}
```

Second, the acknowledge wrapper, further down the same file.

FIND:

```
public void acknowledgeDeviations(List<String> deviationIds) {
    try {
        healthBusiness.acknowledgeDeviations(deviationIds);
    } catch (Exception e) {
        LOGGER.error("Could not acknowledge deviations", e);
    }
}
```

REPLACE WITH:

```
public void acknowledgeDeviations(Aircraft aircraft, List<String> deviationIds) {
    if (aircraft == null) {
        return;
    }
    try {
        healthBusiness.acknowledgeDeviations(aircraft.getId(),
            MgsSession.getInstance().getUser().getLoginName(), deviationIds);
    } catch (Exception e) {
        LOGGER.error("Could not acknowledge deviations", e);
    }
}
```

No new imports: `Aircraft` and `MgsSession` are already imported in this file.

`getLoginName()` is the right name – it is what `HealthTreeNodeLearnThresholdAction` passes for the same parameter.

Step A6 — DeviationTableComposite

`mgs-client/.../preferences/healthalarms/DeviationTableComposite.java`

The Delete Deviation button calls the wrapper that just gained a parameter.

FIND the call inside the Delete button's listener (search for `acknowledgeDeviations`):

```
store.acknowledgeDeviations(ids);
```

REPLACE WITH:

```
store.acknowledgeDeviations(MgsSession.getInstance().getAircraft(), ids);
```

If the composite already holds the aircraft as a field, use that instead of the session – it is the same aircraft either way. Add the `MgsSession` import if it is not there.

Step A7 — an unrelated NPE in the same file

toDtos has an unrelated NPE. The filtering-law parameters loop sits outside the null check immediately above it.

FIND:

```
if (d.getFilteringLaw() != null) {
    dto.setFilteringLawId(d.getFilteringLaw().getId());
    dto.setFilteringLawName(d.getFilteringLaw().getName());
}
StringBuilder params = new StringBuilder();
for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
```

REPLACE WITH:

```
StringBuilder params = new StringBuilder();
if (d.getFilteringLaw() != null) {
    dto.setFilteringLawId(d.getFilteringLaw().getId());
    dto.setFilteringLawName(d.getFilteringLaw().getName());
    for (ContextualParameter p : d.getFilteringLaw().getContextualParameters()) {
```

Then close the extra brace at the end of that loop. A deviation created without a filtering law throws today when the Deviation List tab loads.

Testing phase A

#	Action	Expect
1	Rebuild mgs-business, mgs-client and the server, then redeploy	–
2	Find an indicator whose threshold is LEARNED. Create a deviation on it changing the min or max, add a TE, Apply and OK	Row turns green
3	SELECT * FROM public.relearn_threshold_request ORDER BY startdate DESC LIMIT 5;	A new row, handled = false, history_learning_time = 0
4	Check the deviation is still listed in the Deviation List tab	Still there – the relearn must not remove it (slide 8)
5	Download flights until the learning time is reached	handled = true and learnedvalue filled on that row
6	Delete the deviation from the Deviation List tab	Moves to history, and a second relearn row appears
7	Create a deviation on a FIXED threshold, leaving the type alone	No new relearn row

Step 3 proves the feature. Step 7 is what separates a correct implementation from one that launches a relearn on every deviation regardless of threshold type.

Phase B – the deviated bounds on a learned value

mgs-download/.../download/analysis/health/impl/RedAmberAlarmAnalyzer.java

This is a Part D gap rather than new work, and it is independent of every other step in this document. It is here because it lands in the same method as phase C, so the two are cheaper done together – but it needs nothing from phase A or C and can ship on its own.

Slide 17: *si la Deviation a modifié la borne alors elle s’applique (même si fin d’apprentissage)*. A deviated min or max clamps the learned value even once learning has finished. Part D substitutes a deviated *value*, but when the value comes from `relearnRequest.getLearnedValue()` the deviated bounds are never applied, so a deviation that only narrowed the bounds has no effect at all on a learned threshold.

`LearnedThresholdAnalyzer.adjustThresholdBounds` already does this with the *configured* bounds, at learning time. The same clamp has to run at analysis time with the *deviated* bounds, because the deviation may have been created after the learning completed.

FIND, the line that opens the threshold value chain (added by Part D, edit 4.3):

```
final double thresholdValue;
if (deviation != null && deviation.getThresholdValue() != null) {
```

REPLACE WITH:

```
// BCR-6605 no longer final: a deviated bound may clamp it below
double thresholdValue;
boolean learnedValue = false;
if (deviation != null && deviation.getThresholdValue() != null) {
```

Then FIND, in the learned branch of that same chain:

```
RelearnThresholdRequest relearnRequest = getApplicableRelearnRequest(currentFlight, thresholdConfiguration);
if (relearnRequest != null) {
    thresholdValue = relearnRequest.getLearnedValue();
} else {
```

REPLACE WITH:

```
RelearnThresholdRequest relearnRequest = getApplicableRelearnRequest(currentFlight, thresholdConfiguration);
learnedValue = true; // BCR-6605 this value is subject to the deviated bounds
if (relearnRequest != null) {
    thresholdValue = relearnRequest.getLearnedValue();
} else {
```

Then FIND the closing brace of the whole chain, the line immediately before `int nbValuesAboveThreshold = 0;`

```
} else {
```

```
        throw new IllegalStateException();
    }

    int nbValuesAboveThreshold = 0;
```

REPLACE WITH:

```
    } else {
        throw new IllegalStateException();
    }

    // BCR-6605 a deviated bound clamps the learned value, even when the
    // learning had already finished when the deviation was created
    if (learnedValue && deviation != null) {
        thresholdValue = applyDeviatedBounds(thresholdValue, level, deviation);
    }

    int nbValuesAboveThreshold = 0;
```

And ADD this method beside `getApplicableDeviation`:

```
/**
 * BCR-6605 clamp a learned value with the bounds the deviation set, if it set any.
 *
 * Mirrors LearnedThresholdAnalyzer.adjustThresholdBounds, including the shape of its LOW
 * branch, so a deviated bound behaves exactly like a configured one. The difference is that
 * a deviation's bounds are optional, so both are null-checked - on the configuration,
 * maximumThreshold is a primitive and always present.
 */
private double applyDeviatedBounds(final double value, final ThresholdLevel level,
    final IndicatorDeviation deviation) {
    final Double min = deviation.getMinValue();
    final Double max = deviation.getMaxValue();
    if (min == null && max == null) {
        // The deviation did not touch the bounds
        return value;
    }
    if (level == ThresholdLevel.HIGH) {
        if (max != null && value > max) {
            return max;
        }
        if (min != null && value < min) {
            return min;
        }
    } else if (level == ThresholdLevel.LOW) {
        if (max != null && max > value) {
            return max;
        }
        if (min != null && min < value) {
            return min;
        }
    }
    return value;
}
```

The LOW branch looks wrong and is copied on purpose. In `adjustThresholdBounds` the LOW case uses `maximumThreshold` as a floor and `minimumThreshold` as a ceiling – the reverse of the HIGH case. Whether that is intentional or an old bug, a deviated bound must behave the same way as a configured one, otherwise the same numbers produce different thresholds depending on where they came from. If it is a bug it should be fixed in both places at once, under its own ticket.

Testing phase B:

#	Action	Expect
1	Find an indicator with a LEARNED threshold whose learning has already completed. Note the learned value.	–
2	Create a deviation setting Max well below that learned value. Leave Value empty.	Row turns green
3	Download a flight	Alarms now raise against the deviated max, not the learned value
4	Delete the deviation and download again	Back to the learned value

Step 2 leaving Value empty is the point: before this change that deviation does nothing at all on a learned threshold.

Phase C – a deviation that turns FIXED into LEARNED

Benoit's note puts “type learned *or modified from fixed to learned*” in scope. Phase A covers the first; this covers the second. It is a bigger change than A and B together, because the data model has no place to hold a relearn request for a threshold that is fixed in the configuration, and the learning algorithm reads every one of its parameters off the learned configuration row.

Why it is not a one-liner

When a deviation sets the type to LEARNED on a threshold that the configuration declares FIXED, three separate things have no home:

What is missing	Where
Somewhere to store the relearn request	<code>relearnRequests</code> is declared on <code>LearnedThresholdConfiguration</code> , not on the <code>ThresholdConfiguration</code> base class, so a <code>FixedThresholdConfiguration</code> has no such collection
A way to create it	<code>HealthDao.createRelearnThresholdRequest</code> takes a <code>LearnedThresholdConfiguration</code> parameter
Learning	<code>computeThresholdLearning</code> reads <code>getLearningTime</code> , <code>getMeanFactor</code> , <code>getDeviationCoefficient</code> , <code>getMinimumDeviation</code> ,

parameters to compute with	getMinimumThreshold and getMaximumThreshold off the learned configuration. For this case they live on the deviation instead.
----------------------------	--

So the work is: give the base class the collection, decouple the learning algorithm from the learned configuration class, and teach the analyzer to read a deviation. Part D already did the hardest part of the last one – `IHealthHistoryProvider.getDeviations()` exists and `updateLearnedThresholds` already receives the provider.

One piece of good news. Moving `relearnRequests` up to the base class needs no DDL. The association is unidirectional with the foreign key in `RELEARN_THRESHOLD_REQUEST.THRESHOLD_CONFIGURATION_ID`, and the threshold configuration hierarchy is single-table with a discriminator. Which Java class declares the field does not change a single column.

Step C1 — move relearnRequests to the base class

mgs-jpa-model/.../datamodel/health/monitoring/LearnedThresholdConfiguration.java

FIND and DELETE, together with its getter and setter:

```
/** The relearn requests. */
@OneToMany(cascade = CascadeType.ALL, fetch = FetchType.EAGER, orphanRemoval = true)
@JoinColumn(name = "THRESHOLD_CONFIGURATION_ID")
private Set<RelearnThresholdRequest> relearnRequests = new HashSet<>();
```

Also remove `getRelearnRequests()` and `setRelearnRequests(...)` from this class, and the now-unused `CascadeType`, `FetchType`, `JoinColumn`, `OneToMany`, `HashSet` and `Set` imports if nothing else in the file uses them.

mgs-jpa-model/.../datamodel/health/monitoring/ThresholdConfiguration.java

PASTE the field and both accessors into the base class instead, keeping the annotations exactly as they were. Every existing caller of `getRelearnRequests()` keeps compiling, because `LearnedThresholdConfiguration` inherits it.

Step C2 — let the DAO create one for any threshold

mgs-services-impl/.../service/health/common/ejb/HealthDao.java

FIND:

```
public void createRelearnThresholdRequest(Aircraft aircraft, LearnedThresholdConfiguration learnedThreshold, Date effectiveDate, Date
startFlightDate, long historyLearningTime,
    String comment, String login, Indicator indicator, List<String> indicatorLocations) {
    RelearnThresholdRequest request = new RelearnThresholdRequest(UUID.randomUUID().toString());
    request.setStartDate(effectiveDate);
    request.setStartFlightDate(startFlightDate);
    request.setHistoryLearningTime(historyLearningTime);
    request.setDescription(comment);
    request.setLearnedValue(learnedThreshold.getValue());
    learnedThreshold.getRelearnRequests().add(request);
```

REPLACE WITH:

```
public void createRelearnThresholdRequest(Aircraft aircraft, ThresholdConfiguration learnedThreshold, Date effectiveDate, Date
startFlightDate, long historyLearningTime,
    String comment, String login, Indicator indicator, List<String> indicatorLocations) {
    RelearnThresholdRequest request = new RelearnThresholdRequest(UUID.randomUUID().toString());
    request.setStartDate(effectiveDate);
    request.setStartFlightDate(startFlightDate);
    request.setHistoryLearningTime(historyLearningTime);
    request.setDescription(comment);
    // BCR-6605 null for a fixed configuration deviated to learned: there is no previous learned value
    request.setLearnedValue(learnedThreshold.getValue());
    learnedThreshold.getRelearnRequests().add(request);
```

Check the last line of this method. It calls `dataCollectionService.collectThresholdRelearnRequest(learnedThreshold, ...)`. If that method also declares a `LearnedThresholdConfiguration` parameter, it needs the same widening, or the call needs guarding with an `instanceof`. Open `DataCollectionService` and check before building.

Step C3 — include fixed configurations in the lookup

mgs-business/.../business/health/impl/HealthCommonBusiness.java

In the `getLearnedThresholdIds` helper added in step A2, the `instanceof` filter now has to let a fixed configuration through when the deviation is the thing making it learned.

FIND:

```
for (ThresholdConfiguration threshold : indicatorConfiguration.getThresholdConfigurations()) {
    if (!(threshold instanceof LearnedThresholdConfiguration)) {
        // Only a configured learned threshold can carry a relearn request
        continue;
    }
}
```

REPLACE WITH:

```
boolean deviatedToLearned = ThresholdType.LEARNED.name().equals(deviatedType);
for (ThresholdConfiguration threshold : indicatorConfiguration.getThresholdConfigurations()) {
    if (!(threshold instanceof LearnedThresholdConfiguration) && !deviatedToLearned) {
        // A fixed configuration only relearns when the deviation turns it learned
        continue;
    }
}
```

On the delete path `deviatedType` is null, so `deviatedToLearned` is false and a fixed configuration is skipped – which is right: removing a deviation from a fixed threshold restores a fixed threshold, and slide 17 lists no relearn for that.

Step C4 — a holder for the learning parameters

mgs-download/.../download/analysis/health/impl/LearningParameters.java (new file)

This is the piece that decouples the algorithm from the configuration class. One record, built either from a learned configuration or from a deviation.

```
package com.eurocopter.mgs.download.analysis.health.impl;

import com.eurocopter.mgs.datamodel.health.monitoring.IndicatorDeviation;
import com.eurocopter.mgs.datamodel.health.monitoring.LearnedThresholdConfiguration;
import com.eurocopter.mgs.datamodel.health.monitoring.ThresholdLevel;

/**
 * BCR-6605 the parameters the learning algorithm needs, resolved from the configuration or,
 * when a deviation changed them, from the deviation.
 */
public record LearningParameters(long learningTime, double meanFactor, Double deviationCoefficient,
    Double minimumDeviation, ThresholdLevel thresholdLevel, Double minimumThreshold,
    double maximumThreshold) {

    /** The configured parameters of a learned threshold, with any deviated value taking over. */
    public static LearningParameters of(LearnedThresholdConfiguration configuration,
        IndicatorDeviation deviation) {
        long learningTime = configuration.getLearningTime();
        double meanFactor = configuration.getMeanFactor();
        Double deviationCoefficient = configuration.getDeviationCoefficient();
        Double minimumDeviation = configuration.getMinimumDeviation();
        ThresholdLevel level = configuration.getThresholdLevel();
        Double min = configuration.getMinimumThreshold();
        double max = configuration.getMaximumThreshold();

        if (deviation != null) {
            if (deviation.getLearningTime() != null) {
                learningTime = deviation.getLearningTime();
            }
            if (deviation.getMeanFactor() != null) {
                meanFactor = deviation.getMeanFactor();
            }
            if (deviation.getDeviationCoefficient() != null) {
                deviationCoefficient = deviation.getDeviationCoefficient();
            }
            if (deviation.getMinimumDeviation() != null) {
                minimumDeviation = deviation.getMinimumDeviation();
            }
            if (deviation.getThresholdLevel() != null) {
                level = ThresholdLevel.valueOf(deviation.getThresholdLevel());
            }
            if (deviation.getMinValue() != null) {
                min = deviation.getMinValue();
            }
            if (deviation.getMaxValue() != null) {
                max = deviation.getMaxValue();
            }
        }
        return new LearningParameters(learningTime, meanFactor, deviationCoefficient, minimumDeviation,
            level, min, max);
    }

    /**
     * The parameters of a threshold that is learned only because a deviation says so. Everything
     * comes from the deviation; the fixed configuration has none of these values.
     */
    public static LearningParameters ofDeviationOnly(IndicatorDeviation deviation,
        ThresholdLevel configuredLevel) {
        return new LearningParameters(
            deviation.getLearningTime() != null ? deviation.getLearningTime() : 0L,
            deviation.getMeanFactor() != null ? deviation.getMeanFactor() : 1d,
            deviation.getDeviationCoefficient(),
            deviation.getMinimumDeviation(),
            deviation.getThresholdLevel() != null
                ? ThresholdLevel.valueOf(deviation.getThresholdLevel()) : configuredLevel,
            deviation.getMinValue(),
            deviation.getMaxValue() != null ? deviation.getMaxValue() : Double.MAX_VALUE);
    }
}
```

The `meanFactor` default of 1 matches `LearnedThresholdConfiguration.getMeanFactor()`, which returns 1 when the column is null. The `maximumThreshold` fallback of `Double.MAX_VALUE` means “no upper clamp”, since a deviation that sets `LEARNED` without a max has no bound to apply.

Step C5 — the analyzer walks every threshold, not just learned ones

mgs-download/.../download/analysis/health/impl/LearnedThresholdAnalyzer.java

FIND `updateLearnedThresholds`:

```
for (final ThresholdConfiguration thresholdConfiguration : indicatorEntry.getValue().getThresholdConfigurations()) {
    if (thresholdConfiguration instanceof LearnedThresholdConfiguration) {
        // This is a learned threshold: retrieve latest relearn request if any
        final List<RelearnThresholdRequest> requests = new ArrayList<>(((LearnedThresholdConfiguration)
thresholdConfiguration).getRelearnRequests());
        if (!requests.isEmpty()) {
            // Retrieve latest relearn request
            final RelearnThresholdRequest latestRequest = Collections.max(requests, new RelearnThresholdRequestComparator());
            if (!latestRequest.isHandled()) {
                computeThresholdLearning(aircraft, flight, indicatorEntry.getKey(), (LearnedThresholdConfiguration)
thresholdConfiguration,
                    latestRequest, commonParameters, healthHistoryProvider, mechanicalComponentEntry.getKey().isGroundAcquisition());
            }
            // Only the last request will be processed (no matter its state)
        }
    }
}
```

REPLACE WITH:

```

for (final ThresholdConfiguration thresholdConfiguration : indicatorEntry.getValue().getThresholdConfigurations()) {
    // BCR-6605 the deviation that governs this threshold, if any
    final IndicatorDeviation deviation = deviations.get(
        indicatorEntry.getKey().getId() + "|"
            + thresholdConfiguration.getThreshold().getSeverity().name());

    final LearningParameters parameters;
    if (thresholdConfiguration instanceof LearnedThresholdConfiguration) {
        parameters = LearningParameters.of(learnedConfiguration, deviation);
    } else if (deviation != null
        && ThresholdType.LEARNED.name().equals(deviation.getThresholdType())) {
        // BCR-6605 fixed in the configuration, learned because of the deviation
        parameters = LearningParameters.ofDeviationOnly(deviation,
            thresholdConfiguration.getThresholdLevel());
    } else {
        // Neither configured nor deviated to learned: nothing to do
        continue;
    }

    final List<RelearnThresholdRequest> requests = new ArrayList<>(thresholdConfiguration.getRelearnRequests());
    if (!requests.isEmpty()) {
        // Retrieve latest relearn request
        final RelearnThresholdRequest latestRequest = Collections.max(requests, new RelearnThresholdRequestComparator());
        if (!latestRequest.isHandled()) {
            computeThresholdLearning(aircraft, flight, indicatorEntry.getKey(), parameters,
                latestRequest, commonParameters, healthHistoryProvider, mechanicalComponentEntry.getKey().isGroundAcquisition());
        }
        // Only the last request will be processed (no matter its state)
    }
}
}

```

And fetch the deviations once, at the top of the same method. **FIND:**

```

void updateLearnedThresholds(final Aircraft aircraft, final Flight flight, final Map<HealthMechanicalComponent,
HealthMechanicalComponentConfiguration> latestHealthConfiguration,
    final HealthMonitoringParameters commonParameters, IHealthHistoryProvider healthHistoryProvider) {
    for (final Map.Entry<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> mechanicalComponentEntry :
latestHealthConfiguration.entrySet()) {

```

REPLACE WITH:

```

void updateLearnedThresholds(final Aircraft aircraft, final Flight flight, final Map<HealthMechanicalComponent,
HealthMechanicalComponentConfiguration> latestHealthConfiguration,
    final HealthMonitoringParameters commonParameters, IHealthHistoryProvider healthHistoryProvider) {
    // BCR-6605 every deviation of this aircraft, keyed by indicator id and severity
    final Map<String, IndicatorDeviation> deviations = healthHistoryProvider.getDeviations();
    for (final Map.Entry<HealthMechanicalComponent, HealthMechanicalComponentConfiguration> mechanicalComponentEntry :
latestHealthConfiguration.entrySet()) {

```

New imports in this class: `IndicatorDeviation` and `ThresholdType`.

This uses the same map Part D added, built by `HealthHistoryProvider.getDeviations()`. Note it is not date-filtered here – the learning runs for the deviation that is current, not for a historical session – which is correct, but worth knowing it differs from the date rule in `RedAmberAlarmAnalyzer`.

Step C6 — the algorithm reads the holder

mgs-download/.../download/analysis/health/impl/LearnedThresholdAnalyzer.java

Three methods swap `LearnedThresholdConfiguration thresholdConfiguration` for `LearningParameters parameters`. The bodies are unchanged apart from the accessor names.

Method	Change
<code>computeThresholdLearning</code>	Signature takes <code>LearningParameters parameters</code> . Inside, both <code>thresholdConfiguration.getLearningTime()</code> become <code>parameters.learningTime()</code> (three occurrences: the <code>Math.min</code> , the <code>remainingTime</code> calculation and the <code>extend-learning</code> branch).
<code>completeLearning</code>	Signature takes <code>LearningParameters parameters</code> . <code>getMinimumDeviation()</code> , <code>getDeviationCoefficient()</code> and <code>getMeanFactor()</code> become <code>parameters.minimumDeviation()</code> , <code>parameters.deviationCoefficient()</code> and <code>parameters.meanFactor()</code> . The switch on <code>thresholdConfiguration.getThreshold().getSeverity()</code> needs the severity passed in as a new parameter, since the holder does not carry it.
<code>ajustThresholdBounds</code>	Signature takes <code>LearningParameters parameters</code> . <code>getThresholdLevel()</code> , <code>getMaximumThreshold()</code> and <code>getMinimumThreshold()</code> become the record accessors. This method is public – search for other callers before changing it, and keep an overload taking <code>LearnedThresholdConfiguration</code> if any exist.

Step C7 — the alarm comparison uses the learned value

mgs-download/.../download/analysis/health/impl/RedAmberAlarmAnalyzer.java

Part D's `thresholdValue` chain puts the deviation first, then fixed, then learned. For a fixed configuration deviated to `LEARNED` it currently takes the deviation's own `thresholdValue` – or the fixed value when the deviation did not set one – and never looks at the relearn request. Once learning completes, it should use the learned value.

FIND, inside the `thresholdValue` chain:

```

    } else if (thresholdConfiguration instanceof FixedThresholdConfiguration) {
        thresholdValue = thresholdConfiguration.getValue();

```

REPLACE WITH:

```

    } else if (thresholdConfiguration instanceof FixedThresholdConfiguration) {
        // BCR-6605 a deviation may have turned this fixed threshold into a
        // learned one; use the learned value once the relearn has completed

```

```
RelearnThresholdRequest deviatedRelearn = (deviation != null
    && ThresholdType.LEARNED.name().equals(deviation.getThresholdType()))
    ? getApplicableRelearnRequest(currentFlight, thresholdConfiguration)
    : null;
if (deviatedRelearn != null && deviatedRelearn.getLearnedValue() != null) {
    thresholdValue = deviatedRelearn.getLearnedValue();
} else {
    thresholdValue = thresholdConfiguration.getValue();
}
```

getApplicableRelearnRequest needs widening too. It casts its argument to `LearnedThresholdConfiguration` on its first line to reach `getRelearnRequests()`. After step 1 that cast can go – change `((LearnedThresholdConfiguration) thresholdConfiguration).getRelearnRequests()` to `thresholdConfiguration.getRelearnRequests()`.

Until the learning completes the threshold keeps the fixed configured value, or the deviation's own value if it set one. That is the only sensible behaviour: there is nothing else to compare against, and the alternative – suppressing the alarm entirely while learning runs – would leave the indicator unmonitored for the whole learning period.

Two files still needed for phase C

- Steps C1 and C2 are written against classes not yet seen, so their FIND blocks are from the fragments visible in other files rather than from the real source:
- `ThresholdConfiguration.java` – to place the moved field exactly, and confirm the class is not `@MappedSuperclass` (if it is, the move behaves differently and step C1 changes).
 - `DataCollectionService`, or just the signature of `collectThresholdRelearnRequest` – to know whether step C2 needs it widened as well.

Scale and risk

Step	Module	Risk
A1 – A7	mgs-business, mgs-client	Low. Confined to the deviation screens and the business layer; nothing in the download path changes.
B1	mgs-download	Medium. One method, but it is the alarm comparison. The clamp only runs when a deviation exists and the value was learned.
C1 – move <code>relearnRequests</code>	mgs-jpa-model	Low. No DDL, inherited accessor keeps every caller compiling.
C2 – widen the DAO	mgs-services-impl	Low, once the data-collection signature is known.
C3 – the lookup	mgs-business	Low. Four lines.
C4 – <code>LearningParameters</code>	mgs-download	Low. New file, nothing depends on it yet.
C5 – <code>updateLearnedThresholds</code>	mgs-download	High. This loop runs for every threshold of every aircraft on every download.
C6 – the algorithm	mgs-download	High. Mechanical, but it is the learning algorithm; a wrong accessor silently produces wrong thresholds.
C7 – the comparison	mgs-download	Medium. Confined to the fixed branch.

C5 and C6 touch the code path that decides every red and amber alarm on every aircraft, deviated or not. A regression there is not visible in the deviation screens – it shows up as alarms that should have fired and did not. Worth running the `mgs-gbx-tests-validation` suite before and after, and comparing the alarm counts on a reference aircraft.

On calling `computeThresholdLearning` directly

Benoit's follow-up points at `LearnedThresholdAnalyzer.computeThresholdLearning`. That method is private, lives in `mgs-download`, and takes a `Flight`, a `HealthMonitoringParameters` and an `IHealthHistoryProvider` – none of which exist in the business layer at the moment a deviation is created. It also could not produce a value there: with `historyLearningTime = 0` and no future flights yet, there is nothing to learn from.

The reading that fits both of his messages is that the request must reach that method with the right parameters, which is what phase A does – `updateLearnedThresholds` calls `computeThresholdLearning` for the latest unhandled request on every download. If he means something else, it is worth one line back to him before building anything.